

Evasion Attack

Michele Pio Lacagnina^{a,1}

^aUnipv, Unimi, Unimib

¹michelepio.lacagnina01@universitadipavia.it

March 18, 2026

Abstract

This report presents the work conducted during the LAB session on **Evasion Attack** with a particular focus on the Fast Gradient Sign Method (**FGSM**) and the **Evaluation Metrics for Adversarial Robustness**.

Keywords: FGSM, Adversarial Robustness, Evaluation Metrics

1. INTRODUCTION

Adversarial Attack is a malicious technique that manipulates machine learning models by deliberately providing them deceptive data to cause incorrect or unintended behavior. These attacks exploit vulnerabilities in the model, often through subtle, imperceptible changes to the input data. They challenge the trustworthiness and reliability of AI systems, which can have serious consequences in sensible AI applications.

These kind of attacks can be classified by when they occur in the machine learning lifecycle and by the attacker's level of knowledge about the model. It is generally used a color code classification: White, Black and Gray-box attack.

- **White-box attacks** happen when the attacker has full access to the model's architecture and parameters.
- **Black-box attacks** are more common and involve the attacker having limited or no knowledge of the model's internal workings, instead relying on querying the model and observing its outputs.
- **Grey-box attacks** represents a middle ground, for those cases the attacker possesses only partial knowledge of the model (either the architecture or the weights)

A further distinction includes:

- **Poisoning:** where malicious data is injected into the training dataset. This corrupts the model's learning process, leading to degraded accuracy or deliberate vulnerabilities;
- **Evasion:** when input data is manipulated to deceive an already trained AI model. They are categorized into two subtypes:
 - **Nontargeted:** the goal is to make the AI model produce any incorrect output,
 - **Targeted:** aims to force the AI model to produce a specific, predefined, incorrect output
- **Model Extraction:** an attacker repeatedly queries a deployed model to create a replica of its functionality, thereby compromising the model's security and intellectual property without ever accessing its code,
- **Inference-Related:** exploit a model's output to extract sensitive information or learn about its training data. This includes model inversion, which reconstructs sensitive data from the model's outputs, and membership inference, which determines if a specific data point was used in the training set,
- **Transfer attacks:** involve creating adversarial examples for one AI system and adapting them to attack other, different AI models.

For readability, the paragraphs of the report are organized as follows: the second addresses the math behind the FGSM, the third discusses the work done in the lab, the fourth presents the proposed implementation of the evaluation metrics, the fifth reports the results obtained, and finally, observations and conclusions are drawn.

2. MATH BEHIND FGSM

2.1. Idea

In the paper *EXPLAINING AND HARNESSING ADVERSARIAL EXAMPLES* published by Goodfellow, Shlens and Szegedy in 2015, it is argued that ML models consistently misclassify adversarial examples (altered inputs including deceiving, yet human-imperceptible, features) because of the local linear nature of neural networks. They support this idea suggesting the FGSM method, which consists in finding an adversarial example that maximizes the loss function while still adhering as much as possible to the original input. Thus finding an adversarial example results in solving an optimization problem. As in the original paper, considering an image as input, the problem can be written as:

$$\begin{array}{ll} \text{maximize} & L(\theta, x + \delta, y) \\ \text{subject to} & \|\delta\|_{\infty} \leq \epsilon \end{array}$$

where:

- x is the input image;
- y is the true label;
- $f(x)$ is the model where, since it is already trained, the parameters θ are already fixed;
- $L(\theta, x, y)$ is the loss function

Since this is computationally expensive to solve exactly. FGSM provides a fast approximation using a single gradient step.

2.2. Derivation

As shown during the lab FGSM can be derived as:

1. **Linear Approximation:** Using first-order Taylor expansion around x :

$$L(\theta, x + \delta, y) \approx L(\theta, x, y) + \delta^T \nabla_x L(\theta, x, y)$$

2. **Maximizing the Dot Product:** To maximize the loss, one wants the perturbation δ to be aligned with the gradient direction. Given the constraint $\|\delta\|_{\infty} \leq \epsilon$, the optimal perturbation is:

$$\delta = \epsilon \cdot \text{sign}(\nabla_x L(\theta, x, y))$$

3. **FGSM Update:**

$$x_{\text{adv}} = x + \epsilon \cdot \text{sign}(\nabla_x L(\theta, x, y))$$

With clipping to the valid range:

$$x_{\text{adv}} = \text{clip}(x + \epsilon \cdot \text{sign}(\nabla_x L(\theta, x, y)), x_{\min}, x_{\max})$$

This method works because, being the loss a function of the input images, even a small perturbation ϵ can change the total input of a great deal. Let us consider $\epsilon = 0.02$ applied to an input image of dimension 28x28 pixels, then, if we consider the L_1 norm the final input can be modified up to $784 * 0.02 = 15.68$.

Fig. 1 Shows the results the authors of the seminal paper obtained. The perturbation is so small that to a human eye it results impossible to perceive the difference, yet, the model is deceived.

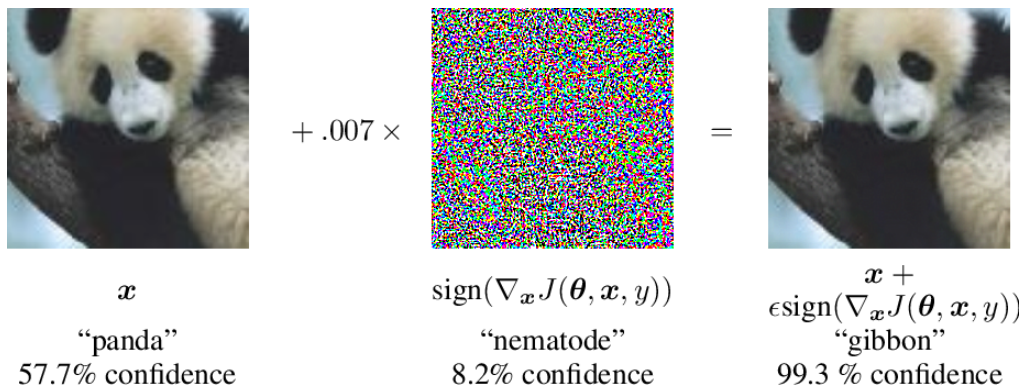


Figure 1. Example of the FGSM effectiveness taken from the original paper

3. WORK DONE IN THE LAB

The work done in the lab included:

- the implementation of a **LeNet** inspired convolutional neural network,
- the usage of a pretrained model the MNIST dataset, to ensure baseline accuracy,
- **FGSM** Attack implementation,
- implementation of prediction and evaluation utilities to:
 - make predictions on images,
 - extract the model probability distribution,
 - evaluate attack success rate,
 - measure model robustness,
 - visualize the above and the obtained classification label
- implementation of targeted FGSM

4. EVALUATION METRICS AND ANALYSIS

To evaluate the adversarial robustness the following metrics are implemented:

1. **Attack Success Rate (ASR)**: the percentage of clean examples that are successfully misclassified by the attack [Code 1](#),
2. **Robust Accuracy**: the percentage of adversarial examples that the model still manages to classify correctly despite the perturbation. [Code 1](#),
3. **Perturbation Norms**:
 - L_∞ : $\|\delta\|_\infty = \max_{i,j} |\delta_{i,j}|$ measures the maximum absolute change applied to any single pixel in the entire image [Code 2](#),
 - L_2 Euclidean: $\|\delta\|_2 = \sqrt{\sum_{i,j} \delta_{i,j}^2}$ measures the standard Euclidean distance, representing the total overall "energy" or magnitude of the perturbation [Code 3](#),
 - L_0 : $\|\delta\|_0 = |\{(i, j) : \delta_{i,j} \neq 0\}|$ counts the total number of pixels that have been altered, regardless of how big or small the change was [Code 4](#),
4. **Confidence changes**: computes the average confidence for the true label on both clean and adversarial examples, as well as the average confidence in the model's adversarial prediction [Code 5](#),
5. **Transferability rate**: a measure to evaluate if adversarial examples of one model also fool another model.

```
def attack_success_rate(model, dataloader, eps):
    """
    Calculates the Attack Success Rate (ASR) of an FGSM attack.
    ASR is the percentage of adversarial examples that successfully
    fool the model into making an incorrect prediction.
    """
    # Set the model to evaluation mode (disables dropout, fixes batch norm)
    model.eval()

    successful_attacks = 0
    total_images = 0

    for imgs, labels in dataloader:
        # Generate adversarial examples for the current batch
        adv, _ = fgsm_attack(model, imgs, labels, eps)

        # Get the model's predictions on adversarial images
        preds, _ = predict_probs(model, adv)

        # 3. Count how many examples were successfully misclassified.
        successful_attacks += (preds != labels.numpy()).sum()

        total_images += labels.size(0)

    return successful_attacks / total_images

def robust_accuracy(model, dataloader, eps):
    """
    Calculates the Robust Accuracy of the model against an FGSM attack.
    """
    # Because an example is either misclassified or correctly classified,
    # robust accuracy is simply 1.0 minus the Attack Success Rate.
    return 1 - attack_success_rate(model, dataloader, eps)
```

Code 1. Proposed implementation of Attack Success Rate

```
def l_inf_norm(model, dataloader, eps):
    """
    Computes the maximum L-infinity norm (maximum per-pixel perturbation)
    across the entire dataset.
    """
    current_max_diff = 0

    for imgs, labels in dataloader:
        # Generate adversarial examples for the current batch
        adv, _ = fgsm_attack(model, imgs, labels, eps)

        # Calculate the L-infinity norm for this specific batch:
        diff = torch.max(torch.abs(imgs - adv))

        # Update the global maximum if the current batch has a larger perturbation
```

```

    if diff > current_max_diff:
        current_max_diff = diff

    return current_max_diff.item()

```

Code 2. Proposed implementation of L_∞ norm

```

def avg_l2_norm(model, dataloader, eps):
    """
    Computes the average L2 (Euclidean) norm of adversarial perturbations
    generated by an FGSM attack across the entire dataset.
    """
    total_l2_norm = 0.0
    total_images = 0

    for imgs, labels in dataloader:
        # Generate adversarial examples for the current batch
        adv, _ = fgsm_attack(model, imgs, labels, eps)

        # Calculate the L2 norm for each image in the batch.
        l2_norms = torch.sqrt(torch.sum((imgs - adv) ** 2, dim=(1, 2, 3)))

        total_l2_norm += torch.sum(l2_norms).item()

        total_images += imgs.size(0)

    avg_l2 = total_l2_norm / total_images
    return avg_l2

```

Code 3. Proposed implementation of L_2 norm

```

def l_zero_norm(model, dataloader, eps):
    """
    Computes the average L0 norm of adversarial perturbations generated by an
    FGSM attack. The L0 norm counts the exact number of pixels that were
    altered by the attack, regardless of the magnitude of the change.
    """
    total_changed = 0
    total_images = 0

    for imgs, labels in dataloader:
        # Generate adversarial examples for the current batch
        adv, _ = fgsm_attack(model, imgs, labels, eps)

        # Calculate the raw difference between the adversarial and original images
        diff = adv - imgs

        # Identify which pixels were changed.
        # Use a tiny numerical tolerance (> 1e-6) instead of strict equality (!= 0)
        # to account for standard floating-point imprecision in PyTorch calculations.
        changed_pixels = torch.abs(diff) > 1e-6

        total_changed += torch.sum(changed_pixels).item()

        total_images += imgs.size(0)

    avg_l0 = total_changed / total_images

    pixels_per_image = imgs.size(1) * imgs.size(2) * imgs.size(3)
    print(f"Average L0 norm: {avg_l0:.2f} (out of {pixels_per_image}).")

    return avg_l0

```

Code 4. Proposed implementation of L_0 norm

```

def calculate_confidence_changes(model, dataloader, eps):
    """
    Evaluates how an FGSM attack affects the model's prediction confidence.

```

```

It calculates three metrics:
1. Clean True Confidence: how confident the model is in the correct label before the attack.
2. Adversarial True Confidence: how confident the model is in the correct label after the attack.
3. Adversarial Predicted Confidence: how confident the model is in its (likely incorrect) new prediction.
"""

model.eval()

# Initialize accumulators the metrics
avg_true_conf_adv = 0.0
avg_adv_pred_conf = 0.0
total_samples = 0

for imgs, labels in dataloader:
    # Move inputs and labels to the designated hardware device
    imgs = imgs.to(device)
    labels = labels.to(device)

    # Get the model's logits for the original, unaltered images
    outputs_clean = model(imgs)
    # Convert raw logits into probabilities using softmax
    probs_clean = softmax(outputs_clean)
    # Identify the class with the highest probability
    preds_clean = torch.argmax(probs_clean, dim=1)

    # Create the perturbed images using the FGSM attack
    adv_imgs, _ = fgsm_attack(model, imgs, labels, eps)

    # Get the model's logits for the newly perturbed images
    outputs_adv = model(adv_imgs)
    # Convert the adversarial logits into probabilities
    probs_adv = softmax(outputs_adv)
    # Identify the model's new predicted class for the adversarial images
    preds_adv = torch.argmax(probs_adv, dim=1)

    for i in range(imgs.size(0)):
        # look up the probability assigned to the true class
        avg_true_conf_clean += probs_clean[i, labels[i]].item()

        # look up the probability assigned to the true class after the attack
        avg_true_conf_adv += probs_adv[i, labels[i]].item()

        # look up the probability assigned to the class the model actually chose
        avg_adv_pred_conf += probs_adv[i, preds_adv[i]].item()

    # running total of the number of images processed
    total_samples += imgs.size(0)

if total_samples > 0:
    avg_true_conf_clean /= total_samples
    avg_true_conf_adv /= total_samples
    avg_adv_pred_conf /= total_samples

print(f"\nConfidence Changes (eps={eps:.3f}):")
print(f" Avg. Confidence in True Label (Clean): {avg_true_conf_clean:.2%}")
print(f" Avg. Confidence in True Label (Adv): {avg_true_conf_adv:.2%}")
print(f" Avg. Confidence in Adversarial Pred.: {avg_adv_pred_conf:.2%}")
print(
    f" Drop in True Label Confidence: {(avg_true_conf_clean - avg_true_conf_adv):.2%}"
)

# Return a dictionary of the final metrics for further analysis
return {
    "avg_true_conf_clean": avg_true_conf_clean,
    "avg_true_conf_adv": avg_true_conf_adv,
    "avg_adv_pred_conf": avg_adv_pred_conf,
}

```

Code 5. Proposed implementation of confidence change statistics

■ TRANSFERABILITY RATE

To further evaluate the robustness of neural networks and the nature of adversarial perturbations, I investigate the phenomenon of *attack transferability*. Transferability refers to the ability of an adversarial example, specifically crafted to deceive one particular network (the Source Model), to successfully fool a completely different, independent network (the Target Model). This property is highly critical from a security standpoint because it demonstrates that adversarial attacks do not merely exploit random, model-specific architectural characteristics, instead, they often manipulate fundamental feature representations that are universally relied upon by various models, making zero-knowledge “black-box” attacks viable in real-world scenarios.

To quantify this vulnerability, I implemented a Transferability Rate metric: the experimental procedure operates by first applying the FGSM attack to the clean MNIST test set using the original Convolutional Neural Network (LeNet) as the white-box Source Model, then, the resulting adversarial images are fed directly into a structurally distinct Target Model—in this case, a custom Mini-ResNet (19706 parameters) architecture featuring batch normalization and residual skip connections (Code 6).

This model is trained (Code 7) on the clean images for 3 epochs using Adam optimizer and Negative Log Likelihood loss, which together with the `log_softmax` (output of the network’s final layer) allow to compute the standard cross-entropy loss required for multi-class classification. The following table shows the result of the training phase.

Table 1. Target model training summary (3 epochs)

Epoch	Loss	Accuracy	Time (s)
1/3	0.9217	79.42%	143.4
2/3	0.1812	96.84%	132.5
3/3	0.1081	97.61%	128.5

The Transferability Rate (Code 8) is subsequently calculated as the absolute percentage of these source-crafted adversarial images that successfully force the target ResNet to output an incorrect classification. By evaluating this metric across multiple ϵ values, it can be measured the extent to which gradient-based evasion attacks transcend specific model architectures.

```
class ResidualBlock(nn.Module):
    """A standard ResNet Block with a skip connection."""

    def __init__(self, in_channels, out_channels, stride=1):
        super(ResidualBlock, self).__init__()

        # First convolutional layer of the block
        self.conv1 = nn.Conv2d(
            in_channels,
            out_channels,
            kernel_size=3,
            stride=stride,
            padding=1,
            bias=False,
        )
        self.bn1 = nn.BatchNorm2d(out_channels)

        # Second convolutional layer
        self.conv2 = nn.Conv2d(
            out_channels, out_channels, kernel_size=3, stride=1, padding=1, bias=False
        )
        self.bn2 = nn.BatchNorm2d(out_channels)

        # The Skip Connection (Identity Mapping)
        self.shortcut = nn.Sequential()
        # If dimensions change (due to stride or channel increase), use a 1x1 conv to match them
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv2d(
                    in_channels, out_channels, kernel_size=1, stride=stride, bias=False
                ),
                nn.BatchNorm2d(out_channels),
            )

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))

        # Add the skipped input back into the output
        out += self.shortcut(x)
        out = F.relu(out)
        return out
```

```

class MiniResNet(nn.Module):
    def __init__(self):
        super(MiniResNet, self).__init__()

        # Initial Convolution
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, stride=1, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(16)

        # Residual Layers
        self.layer1 = ResidualBlock(16, 16, stride=1)
        self.layer2 = ResidualBlock(16, 32, stride=2)

        # Final Fully Connected Output
        self.fc = nn.Linear(32, 10)

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.layer1(out)
        out = self.layer2(out)

        # Global Average Pooling squashes the spatial HxW dimensions down to 1x1
        out = F.adaptive_avg_pool2d(out, (1, 1))
        out = torch.flatten(out, 1)
        out = self.fc(out)

        return F.log_softmax(out, dim=1)

# Initialize the target model and move to device
target_model = MiniResNet().to(device)
print(target_model)
print(
    f"\n Target Mini-ResNet created with {sum(p.numel() for p in target_model.parameters());} parameters"
)

```

Code 6. Implementation of the ResNet model (19706 parameters)

```

def train_target_model(model, trainloader, device, epochs=3, learning_rate=0.001):
    """
    Trains the target model on the clean MNIST dataset.
    """
    # Setup Adam optimizer and loss function
    optimizer = optim.Adam(model.parameters(), lr=learning_rate)

    print(f"Starting Training for {epochs} Epochs...")

    for epoch in range(epochs):
        model.train() # Put the model in training mode (enables BatchNorm)
        running_loss = 0.0
        correct_predictions = 0
        total_samples = 0

        start_time = time.time()

        for batch_idx, (imgs, labels) in enumerate(trainloader):
            # Move data to the correct device
            imgs = imgs.to(device)
            labels = labels.to(device)

            # Zero out the gradients from the previous batch
            optimizer.zero_grad()

            # Forward pass (make predictions)
            outputs = model(imgs)

            # Calculate the loss
            # Use nll_loss because the model outputs log_softmax
            loss = F.nll_loss(outputs, labels)

            # Backward pass (compute gradients)
            loss.backward()

            # Update the model weights
            optimizer.step()

```

```

# Tracking Metrics
running_loss += loss.item() * imgs.size(0)

# Calculate accuracy for this batch
preds = torch.argmax(outputs, dim=1)
correct_predictions += (preds == labels).sum().item()
total_samples += labels.size(0)

# Calculate average metrics for the epoch
epoch_loss = running_loss / total_samples
epoch_accuracy = correct_predictions / total_samples
epoch_time = time.time() - start_time

print(
    f"Epoch [{epoch+1}/{epochs}] "
    f"| Loss: {epoch_loss:.4f} "
    f"| Accuracy: {epoch_accuracy:.2%} "
    f"| Time: {epoch_time:.1f}s"
)

print(" Target Model Training Complete!")

# Ensure the model is on the right device
target_model = target_model.to(device)

# Call the training loop
train_target_model(target_model, trainloader, device, epochs=3)

```

Code 7. Implementation of the target model training loop

```

def transferability_rate(source_model, target_model, dataloader, eps, device="cpu"):
    """
    Calculates the Transferability Rate of an adversarial attack.
    It generates adversarial examples using a white-box Source Model,
    and tests if those exact same images successfully fool a black-box Target Model.
    """
    # Set both models to evaluation mode
    source_model.eval()
    target_model.eval()

    successful_transfers = 0
    total_images = 0

    for imgs, labels in dataloader:
        imgs = imgs.to(device)
        labels = labels.to(device)

        # Use the source model's gradients to make the adversarial noise
        adv_imgs, _ = fgsm_attack(source_model, imgs, labels, eps)

        # Pass the adversarial images into the separate target model
        target_outputs = target_model(adv_imgs)

        # Get the predicted classes from the target model
        target_preds = torch.argmax(target_outputs, dim=1)

        # Count the misclassifications
        # A transfer is successful if the target model's prediction does not equal the true label
        successful_transfers += (target_preds != labels).sum().item()
        total_images += labels.size(0)

    # Calculate the final rate
    transfer_rate = successful_transfers / total_images

    print(
        f"Transferability Rate (epsilon={eps:.3f}): {transfer_rate:.2%} of attacks fooled the target model."
    )
    return transfer_rate

```

Code 8. Implementation of the transferability rate metric

5. RESULTS OBTAINED

The metrics introduced in the previous paragraphs have been computed using four different values of ϵ , as reported in Table 2. As expected, the results demonstrate that as the perturbation budget increases, the model finds it significantly harder to predict the correct class. For instance, under a minimal attack where $\epsilon = 0.01$, the model still retains a relatively high Robust Accuracy of 87.43%, with an Attack Success Rate (ASR) of only 12.57%. However, when the attack strength is maximized at $\epsilon = 0.3$, the model’s performance completely collapses: Robust Accuracy drops to 0.14%, and the ASR increases to 85.82%.

The perturbation norms give a physical picture of how the attack alters the images. L_0 norm, stays completely flat at exactly 311.92 across all tests, this means that no matter how weak or strong the attack is, FGSM always manipulates the exact same number of pixels—touching roughly 81% of the 28×28 image every single time. What actually changes as we increase the attack strength (ϵ) is the *intensity* of the noise applied to those specific pixels.

On the other hand, the line plot (Fig. 2) illustrates how the FGSM attack effects the model’s internal certainty across the different ϵ values. The blue line establishes the baseline, demonstrating that the model is certain (approaching 100% confidence) when evaluating clean, unperturbed images. As the attack strength increases, the model’s confidence in the true label (orange line) decreases rapidly. However, the most interesting insight is provided by the green line (Adversarial Prediction Confidence): the attack does not only make the model unsure or confused, instead, it actively forces the model to become highly confident in its incorrect answers.

Moreover, Fig. 3 shows an example coming from the dataset. An augmentation of the perturbation is provided to make the modified pixels more visible.

Finally, the empirical results of the transferability experiment (Table 3) reveal the vulnerability to black-box attacks. When evaluating the adversarial examples crafted from the source LeNet model against the independently trained Mini-ResNet target, we recorded an alarming Transferability Rate of 90.25% at a maximum perturbation budget of $\epsilon = 0.3$. This metric indicates that nearly 97% of the malicious inputs successfully deceived a completely unseen and fundamentally different architecture. Crucially, the target model’s modern structural advancements—specifically its residual skip connections and batch normalization—failed to provide any inherent defense against the transferred noise. This exceptionally high transfer rate proves that the FGSM perturbations do not merely overfit to the specific, isolated decision boundaries of the source network. Instead, they actively corrupt fundamental, universal feature representations of the input space that both models rely on, underscoring the critical security threat posed by zero-knowledge evasion attacks.

Table 2. Results obtained

Metric	$\epsilon = 0.0$	$\epsilon = 0.1$	$\epsilon = 0.2$	$\epsilon = 0.3$
Attack success rate	0.0088	0.1257	0.5124	0.8582
Robust accuracy	0.9912	0.8743	0.49	0.14
L_∞	2.38e-07	0.32	0.65	0.97
L_2	8.76e-07	5.54	10.97	16.32
Avg. L_0	0.0	311.92	311.92	311.92
Avg. Confidence in True Label (Clean)	98.95%	98.95%	98.95%	98.95%
Avg. Confidence in True Label (Adv)	98.95%	82.53%	33.51%	13.82%
Avg. Confidence in Adversarial Pred.	99.45%	90.65%	58.02%	46.47%
Drop in True Label Confidence	0.0%	16.42%	65.44%	85.13%

Table 3. Transferability rate across attack strengths

ϵ	Transferability Rate
0.000	2.24%
0.100	67.61%
0.200	89.98%
0.300	90.25%

Note

It is important to note that the L_∞ values reported in the table reflect the model’s normalized computational space rather than the raw $[0, 1]$ pixel scale. Because the MNIST dataset was standardized during preprocessing using a standard deviation of $\sigma = 0.3081$, an expected maximum pixel change of $\epsilon = 0.3$ translates to a magnitude of $0.3/0.3081 \approx 0.9737$ within the normalized tensors. This confirms that the L_∞ norm is perfectly matching the allowed ϵ budgets. Moreover, since the attack is pushing those same pixels intensity value in this space, the overall visibility of the noise (measured by the L_2 norm) grows steadily from 5.54 up to 16.32.

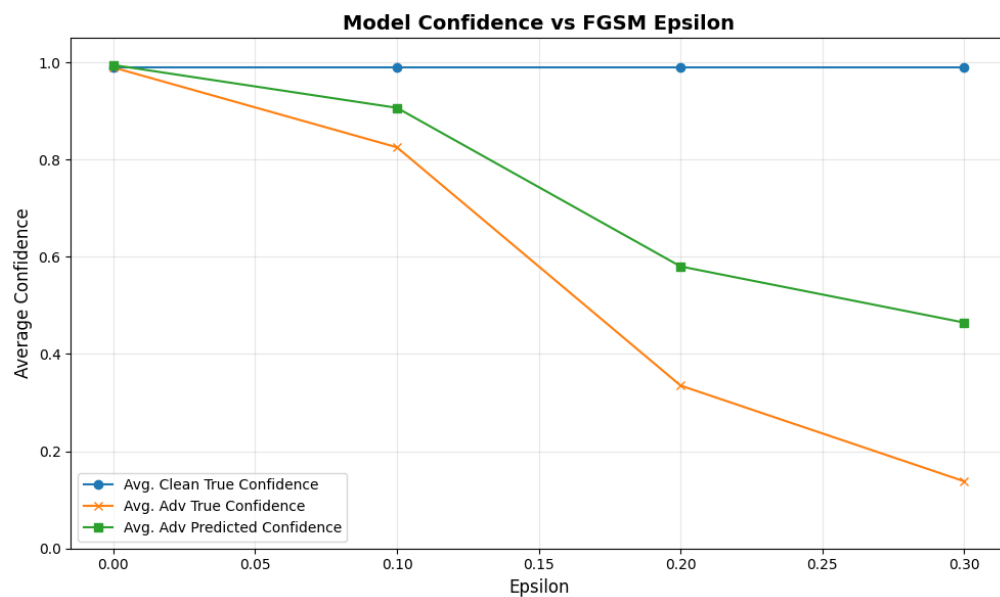


Figure 2. Model confidence trend for increasing values of ϵ

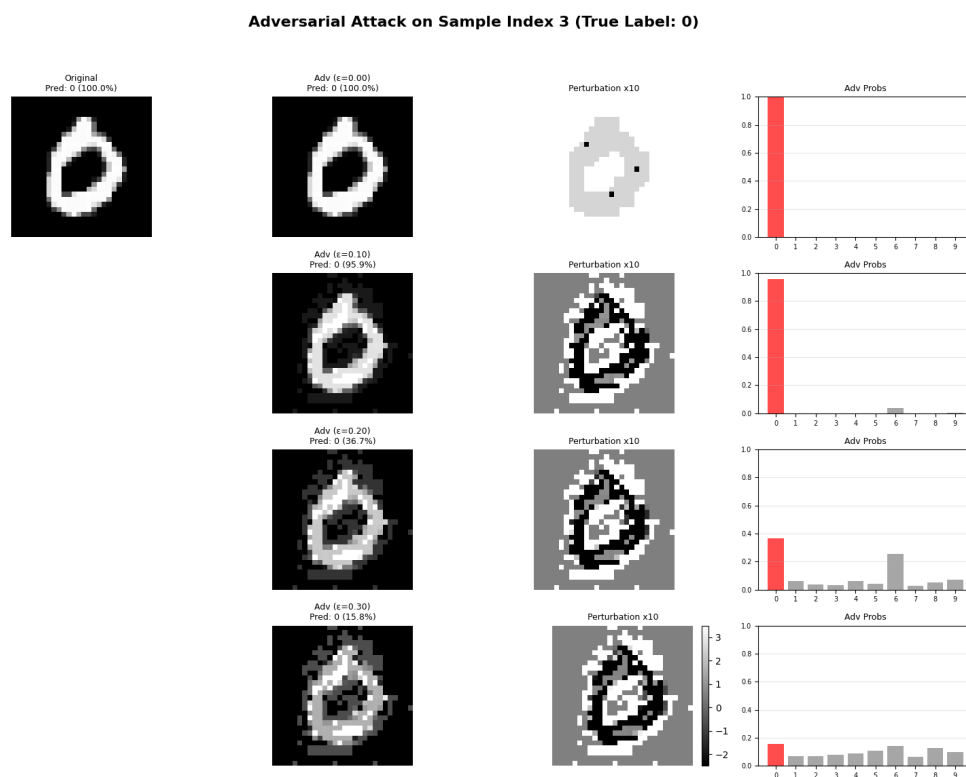


Figure 3. Visualization of the perturbation along with the label's probability distribution